Strathmore University

School of Computing and Engineering Sciences

Project Title: IoT Environment Monitoring System

BICS: Embedded Systems and IoT

Lecturer: Mr. Solomon Itotia

Date: 11/06/2026

Repository: https://github.com/quantanmreaper/MicroPython-Lab-ICS-4D-Group-1.git

Group 1

167077 - Mepani Bhavin Ramesh

161088 - Muhia Robby Mwangi

169649 - Kimathi Austin Muthomi

167999 - Samuel Mwesigwa

166529 - Hope Wanjohi

166991 - Omwanda Philip Maxwell

Chapter 1: Abstract

This project implements a complete Internet of Things (IoT) pipeline which measures temperature and humidity using a DHT22 sensor connected to a TTGO LoRa32 board. The board runs MicroPython, connects to WiFi, and publishes the readings as JSON messages over the MQTT protocol to a public broker (broker.hivemq.com). A Python subscriber running on our PC receives these messages and stores each reading, with a timestamp, into an SQLite database. The system was first prototyped in the Wokwi browser simulator and then deployed to physical TTGO hardware.

Chapter 2: Objectives

Our system was required to:

a) Use **MicroPython** as the firmware language.

b) Use a **TTGO** board as the target hardware.

c) Read **temperature** and **humidity** from a **DHT22** sensor.

d) Connect to a network over **WiFi**.

e) **Publish** the data through **MQTT**.

f) Send the data as **JSON** payloads.

g) **Receive** the messages on a **PC subscriber**.

h) **Store** readings into an **SQLite** database.

i) Produce **screenshots** and **documentation**.

j) Publish the **source code to GitHub**

2.1 Hardware Note

Our lecturer, Mr. Itotia instructed us to use TTGO, instead of the specified generic ESP32. It still uses the same approach, but we had to make sure to check the datasheet before flying them and use WiFi for data transmission. We thus used the **TTGO LoRa32** as the target board. The main difference between the two was the **pin map:** on the TTGO, we used GPIO23 directly addressing the lecturer's note.

2.2 System Architecture

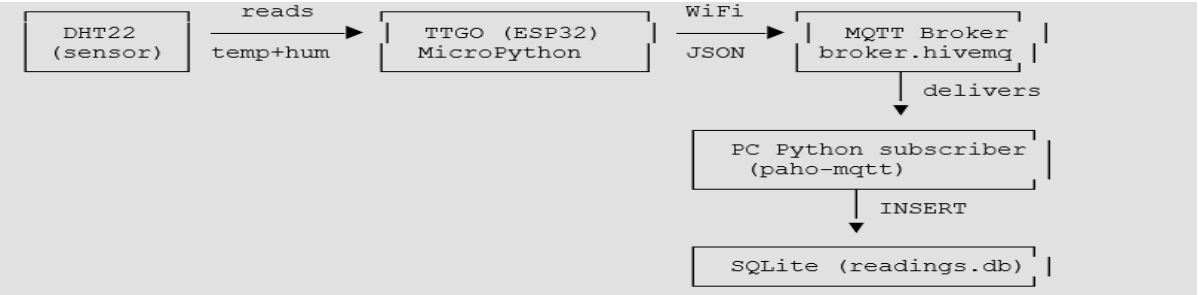

Figure 2.1 System Architecture

Data flow:

1. The DHT22 measures temperature and humidity.
2. The TTGO (MicroPython reads both values from the sensor's DATA pin.
3. The values are packed into a JSON string, e.g. {"temperature": 24.5, "humidity": 60.0}.
4. The board connects to WiFi and publishes the JSON to the topic *university/group1/dht22* via the MQTT broker
5. The broker forwards the message to all subscribers of that topic.
6. The PC subscriber receives the JSON, parses it, and inserts a timestamped row into the SQLite database.

2.3 Components & Tools

Table 2.1 highlights the components and tools which were used in the completion of the lab

Table 2.1 Components and Tools

Component	**Role**	**Notes**
TTGO LoRa32 (ESP32)	Microcontroller / "brain"	WiFi used; LoRa unused
DHT22	Temperature + humidity sensor	3 connections only
MicroPython	Firmware language on the board	umqtt.simple, dht, network
WiFi	Data transport	ESP32 is 2.4 GHz only
MQTT	Messaging protocol	publish/subscribe
HiveMQ broker	Message router	broker.hivemq.com:1883 (public)
JSON	Payload format	json.dumps / json.loads
paho-mqtt	PC MQTT client library	pip install paho-mqtt==1.6.1
SQLite	Database	single file readings.db

Wokwi	ESP32 simulator	used for prototyping
PyCharm + mpremote	Editor + tool to upload/run on the board	python -m mpremote over USB

2.4 Methodology

Wiring: The DHT22 sensor was connected to the TTGO ESP32 using 3.3 V (VCC), GND, and GPIO23 (DATA). During simulation, GPIO15 was used in Wokwi.

Firmware: A MicroPython program connected the board to WiFi and an MQTT broker, read temperature and humidity from the DHT22 sensor every 5 seconds, formatted the readings as JSON, and published them to an MQTT topic.

Data Storage: A Python MQTT subscriber running on the laptop received the sensor readings and stored them in an SQLite database together with timestamps for later retrieval and analysis.

Simulation and Prototype: We first simulated the solution on Wokwi and then went on ahead to build the actual physical hardware and IoT solution.

Prototype: We soldiered the TTGO board ourselves in order to for the pins to be used for connection. The methodology for the setup of the system is shown in Figure *2.2*

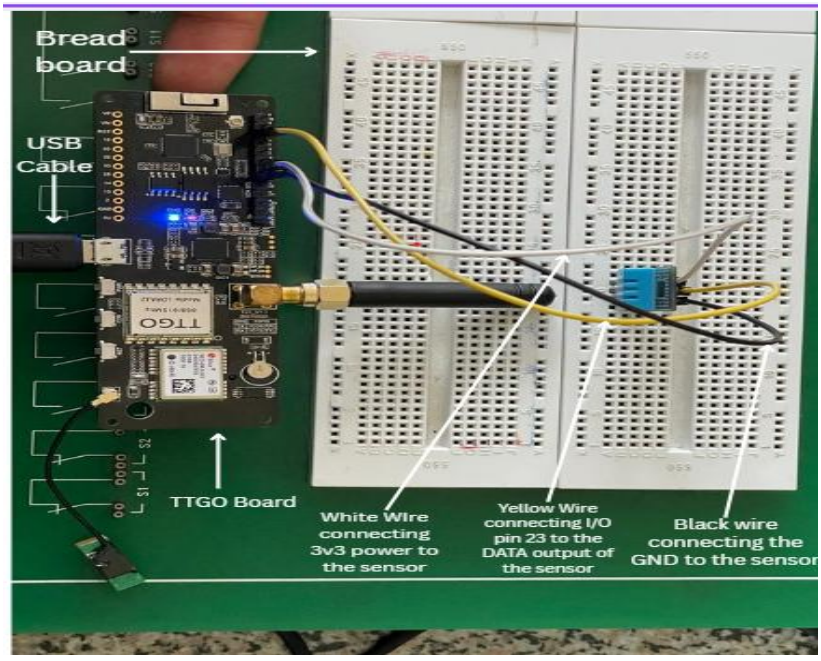

Figure 2.2 Image of Prototype

2.5 Results

The system was able to successfully transmit environmental data from the DHT22 sensor to the SQLite database through MQTT. The Wokwi simulation we had done first verified correct WiFi connectivity, MQTT communication, and JSON message publishing before deployment to physical hardware. On the TTGO board, live temperature and humidity readings were received and stored successfully. Changes in environmental conditions, such as breathing near the sensor, produced immediate increases in humidity values, confirming end-to-end functionality shown in Figure 2.3.

	id	temperature	humidity	timestamp
1	1	24	40	2026-06-10 11:49:15
2	2	24	40	2026-06-10 11:49:20
3	3	24	40	2026-06-10 11:49:26
4	4	24	40	2026-06-10 11:49:31

Figure 2.3 A portion of data received

2.6 Testing and Debugging

The errors in discussion we faced are shown in Figure *2.4* and Figure 2.5, which are then described concisely in

Table *2.2*. The table is also inclusive of how we resolved such errors.

Figure 2.4 Sensor/Read error

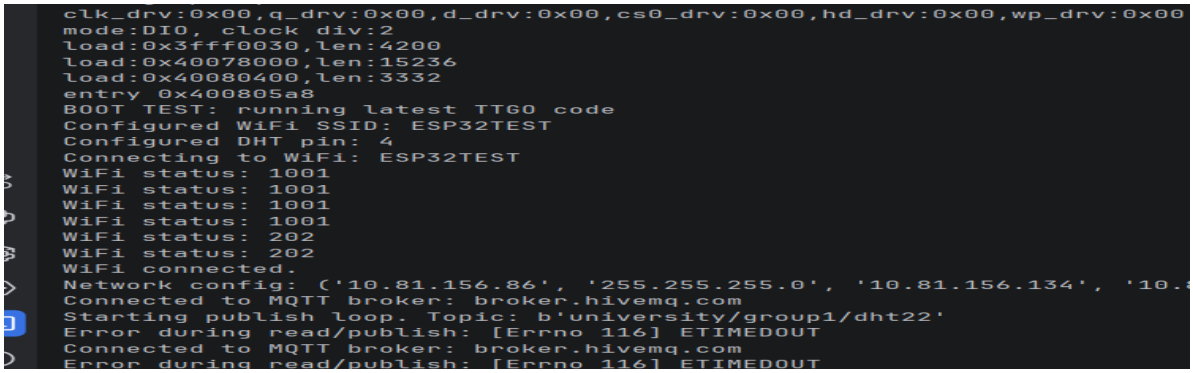

Table 2.2 Testing and Debugging

Figure 2.5 Sensor Read/Publish error

Problems	Cause	Solution
Sensor read error timeout (Err no 116)	The sensor we were working with was faulty	We replaced the DHT22 sensor with a different sensor which resolved the error
Sensor read/publish error (Err no 116)	Incorrect wiring / Pin configuration	After changing pins from GPIO4 to GPIO 23, this error was resolved.

2.7 Conclusion

The project successfully implemented the IoT system using the TTGO board, DHT22 sensor, MQTT communication, and SQLite storage. The sensor readings were collected as needed, transmitted wirelessly, and stored for analysis, showing the practical application of IoT technologies. This was a good project for our group as it has helped us as well in understanding the process of communication in IoT. It will also help us significantly in developing our semester project.

2.8 Appendix

This section covers a number of appendices significant to the lab.

2.8.1 Appendix 1 Wokwi Prototype Simulation

Figure *2.6* shows the Wokwi simulation of the prototype we designed first to get a better understanding of the Lab

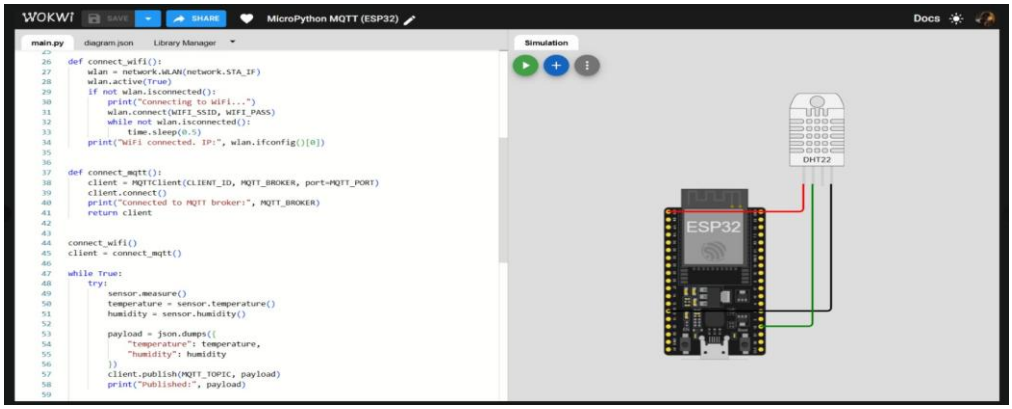

Figure 2.6 Wokwi Prototype

2.8.2 Database Setup

Figure *2.7* highlights the setup file for the creation of our database for storage of readings.

```
#ICS GROUP 4D GROUP 1 EMBEDDED SYSTEMS AND IOT
import sqlite3

DB_FILE = "readings.db"


def create_database():
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()

    cursor.execute(
        """
        CREATE TABLE IF NOT EXISTS readings (
            id          INTEGER PRIMARY KEY AUTOINCREMENT,
            temperature REAL,
            humidity    REAL,
            timestamp   TEXT
        )
        """
    )

    conn.commit()
    conn.close()
    print("OK: database '{}' is ready with table 'readings'.".format(DI


if __name__ == "__main__":
    create_database()
```

Figure 2.7 Database Setup

2.8.3 Database Query

Figure *2.8* highlights our process of querying the database to retrieve the readings stored within it. The figure as well shows the output of the stored readings within the database.

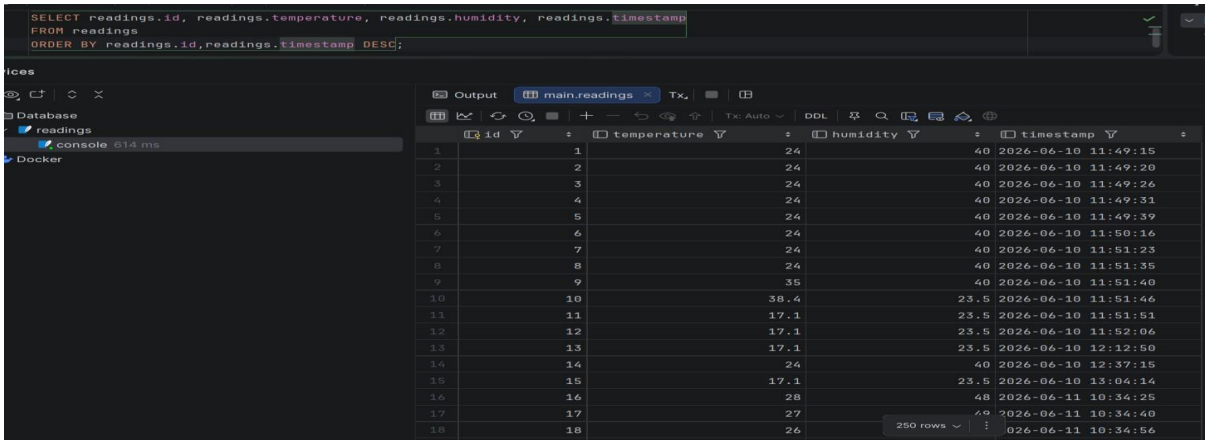

Figure 2.8 Database Query and Output

2.8.4 Installation of Required Libraries

Figure 2.9 below highlights the installation of the required libraries we needed for the MicroPython lab.

Figure 2.9 Installation of Required Libraries

2.8.5 Group Photo

Finally, in Figure 2.10, is our group photo of us working together in the lab.

Figure 2.10 Group Photo

Our members (from front to back):